LEAP Project: Guide for Instructors Acting as Customer

Playing the business for another classroom's delivery team

For faculty use, not for distribution to students

Table of Contents

1. Why you're reading this

For the LEAP Project, each delivery team (a classroom of graduates) builds a trading platform over 11 weeks and runs it as a real agile project: their own architecture, their own backlog, and their own **customer**. That customer is you, an instructor, playing the business that commissioned the platform.

You will act as customer for a classroom that is **not the one you teach**. This is deliberate. It reproduces two things every graduate will meet on a real engagement:

1. **A customer who isn't always available.** You have a day job. They have to plan around you, come prepared, and use the time they get well.
2. **A customer they don't already know.** They can't read your face the way they read their own instructor's. They have to ask properly, listen properly, and confirm what they heard.

This guide covers what the project is, what your role is, and how to play it. It is short on purpose. The one part that needs your attention before Day 1 is **Appendix B**, the shared answers the faculty should agree so that every classroom hears a consistent story.

---

2. What the project actually is

2.1 The brief the students are working from

At kickoff, every team is issued the same document: the **LEAP Project Business Requirements Specification** for a Direct Trading Platform. The premise:

> A financial firm currently serves retail clients through an advised, phone-based dealing desk. Clients keep asking for the same thing: to see their own positions and place their own trades, in the moment, without waiting for a call back. The firm has approved investment in a **direct-to-consumer trading platform**: a retail client signs in, places an order, and sees it reflected in their holdings, with **no dealer in the loop**.

The significant shift, and the thing the students keep coming back to: today a person stands behind every trade and can catch a mistake before it settles. In the new platform, that judgement has to be **designed into the system**.

There is no existing platform to migrate from. This is a new capability.

2.2 What the platform has to do

The spec is written as **business outcomes**, not features to code. In outline:

- **Registration and secure sign-in.** A client sees only their own positions, cash and history.
- **Order placement and execution.** A client submits a buy or sell; the platform checks it against the firm's trading rules (enough cash or stock, instrument currently tradable), records the accepted order as a firm commitment, then prices and executes it separately. Execution can fail or lag without losing the record of intent.
- **Positions, cash and blotter.** A client can see current holdings, cash balance, and a chronological history of their orders and fills.
- **Market data and pricing.** Orders are priced against current market quotes across equities (UK/US and Indian markets), foreign exchange, and crypto assets.
- **Record-keeping and audit.** Every accepted order, pricing decision and resulting change to cash or holdings is permanently recorded, attributable to a client and a time, and survives a restart or a bad deployment. Any

trade's full lifecycle can be reconstructed for audit or a client dispute, without relying on anyone's memory.

- **Reporting and insight.** The business can analyse trading activity over time (by instrument, by period, by client segment) **without that analysis competing with live client trading**.
- **One differentiator (BR-18).** The team proposes and builds one additional capability that extends the platform's value, and justifies why they chose it.

Plus service-wide expectations: a displayed balance that is always correct (including straight after a failure), no trade record ever lost or duplicated, responses that feel immediate, financial-grade security, trade records that cannot be altered after the fact, and a UI that works for a non-technical client with no training.

2.3 Who the business is: the three personas

You may be asked to speak as any of these. They are your reference points for "what the business wants".

- **Joanna, self-directed investor.** Trades a personal portfolio around a full-time job. Checks positions in the evening and around news events. Wants to place a trade in under a minute and trust that what she sees on screen is her real position. Not a professional; no patience for jargon or long confirmation flows. **She is the user.**
- **David, head of trading operations.** Never touches the client UI, but is accountable for every trade it produces. For any disputed trade he needs to say exactly what happened and when, **without asking engineering to dig through logs**.
- **Priya, commercial analyst.** Reports monthly to the executive committee on trading volumes and client activity. Needs reliable historical data, and her queries **must not compete with live trading** for capacity.

Above all three sits the **Programme Sponsor** (Head of Digital Distribution), who owns the business case and the final sign-off. When in doubt, that is the voice you are speaking in.

2.4 The 11-week shape

Teaching runs Monday to Thursday; **Friday is project day** (Sprints 10 and 11 are full weeks). Teams build with whatever that week has just taught them.

Week	Taught focus	What the team produces on project day
1	DevOps fundamentals	Kickoff: candidate architecture, delivery plan, risk list, pitched to a stakeholder panel
2	Agile/Scrum, PR workflow, CI/CD, IaC, OWASP, SRE	**First customer meeting**; a product backlog drawn from the spec
3	Data systems, Postgres, SQL, modelling, NoSQL	Trade database schema / storage decisions
4	Python, pandas, EDA, APIs, ETL	Analytics “dashboard” (3 business insights) + ETL pipeline
5	Java, OOAD, UML, SOLID, TDD	Domain engine
6	Spring Boot, REST, OpenAPI, JWT	Trade REST API
7	Kafka, event-driven architecture, DevSecOps	Event backbone
8	IAM / zero trust, JS/TypeScript, Node, NestJS	Auth service

Week	Taught focus	What the team produces on project day
9	HTML/CSS, Angular	Trading UI, wired to their real backend
10	(no teaching) applied project week	Close the gap against the spec; optional BR-18
11	AWS, deployment automation	Cloud deployment and final showcase

2.5 Where you fit

You are needed **live** in a handful of sprints, and reachable for short questions in between:

- **Sprint 2, the first meeting.** The big one. They plan it, hold it in another room, and build their backlog from what you tell them. Expect broad, sometimes unfocused questions.
- **Sprint 3, a quick check.** Usually about data: how long records really need to be kept, what "reporting" actually means to the business.
- **Sprint 4, a quick check.** Which business insights actually matter to you (Priya's world), so they don't build a dashboard around the wrong three.
- **Sprint 9, a look at the UI.** They show you the working interface, ideally place an order in front of you, and take back your honest reaction, including confusion.
- **Sprint 10, a prioritisation call.** If they can't finish everything, they ask you what matters most. You've sat through their demos; you get to say.
- **Sprint 11, the showcase.** You may be on the panel. Different mode: you're being shown, not interviewed.

Between those, expect the occasional "ten-minute question". Answering some and deferring others is part of the simulation (Section 4).

3. Your role as customer

3.1 The one-line version

You own the problem. They own the solution. You are a domain and business stakeholder, not a technical reviewer. You never tell them how to build anything; you tell them what the business needs and why, and whether what they've shown you would actually work for Joanna, David or Priya.

3.2 What you do

- Explain the business, the personas and the pressures behind the requirements, in business language.
- Answer questions about **intent**: what a requirement is for, what "good" looks like, what would count as the platform failing the business.
- Make **business trade-offs** when asked: what matters most, what could slip, what is non-negotiable.
- React honestly to what they demo, as the business would, not as a marker would.
- Fill gaps the spec deliberately leaves open (retention period, segments, which insights, the BR-18 appetite), using the shared answers in Appendix B so every classroom hears the same thing.

3.3 What you don't do

- **Don't specify technology or architecture.** No databases, frameworks, message buses, cloud services, patterns. If asked "should we use Kafka / Postgres / microservices?", the honest customer answer is "I don't know what that is, that's your call. What I need is..." and restate the outcome.
- **Don't design their data model, API or UI for them.** React to what they propose; don't hand them the answer.
- **Don't mark, coach or rescue.** You are not assessing them and not their instructor for this. If they ask a vague question, they get a vague answer (Section 4). Their own instructor handles teaching and assessment.
- **Don't invent hard facts on the spot.** If it isn't in the spec or Appendix B and it matters, say "good question, let me confirm and come back to you," then check with the faculty so the answer stays consistent across rooms.
- **Don't volunteer the whole picture.** Answer what's asked. Let them discover the rest by asking better questions.

4. How to behave as a customer

This is the part that makes the exercise work. The goal is a realistic customer, not a helpful oracle and not an obstacle.

4.1 Be a real stakeholder, not their instructor

You're a busy professional they've just met. Be courteous and engaged, but:

- Don't fill silences for them. If they stop asking, the meeting is over.
- Don't rephrase their weak question into a good one and then answer the good one. Answer what they actually asked.
- Don't reference the teaching. You've never heard of the module they did on Tuesday.
- Stay in the business world: talk about clients, trades, disputes, the exec committee, risk and compliance breathing down your neck, not systems.

4.2 Be available, but not always

You have a day job. In practice:

- The **Sprint 2 meeting** is booked and you turn up to it properly.
- Ad-hoc questions later in the programme: sometimes you have ten minutes now; sometimes it's "not today, catch me at the start of Friday," or "I'm tied up until Thursday." Vary it. A team that leaves everything to a last-minute question should feel that cost.
- If they didn't prepare, don't compensate by working harder than they did. A meandering meeting is allowed to meander.

4.3 Be vague on occasion, deliberately

Real customers don't hand over clean specs. When a question invites it, give an answer that is **directionally true but underspecified**, and let them come back to pin it down:

- "How long do we need to keep trade records?" → "Basically forever, as far as I'm concerned, Compliance would never forgive us for losing one." (They

should follow up for a number. If they do, give them the Appendix B figure. If they don't, that's on them.)
- "What do you mean by reporting?" → "Priya needs to know what's going on in the book, what's trading, who's active. She reports up to the exec every month." (Leaves them to ask which cuts, how fresh, how far back.)
- "What instruments at launch?" → "Shares, currencies, a bit of crypto." (The spec has the detail; make them go and read it, or ask again.)

Vagueness is a prompt for a better question, **not** a refusal, and not a contradiction of the spec. Don't be vague about a hard requirement that's already written down; point them to it.

4.4 Say yes to features they offer you

When a team pitches something that isn't strictly required ("we could add price alerts", "we could show a performance chart", "we could let them set up recurring buys"), be an **enthusiastic customer**. "Oh, that sounds great, yes, could you?"

This is intentional. Scope creep from a keen customer is one of the lessons. Their job is to manage it: to note it in the backlog, size it, prioritise it against the Musts, and push back themselves if it's a distraction. Your job is to want it. Don't manage the scope for them, and don't warn them off. (BR-18 explicitly asks for one such capability, but they should still be choosing and justifying, not just building everything you got excited about.)

4.5 Reward good practice by responding to it, not praising it

- A team that comes with an agenda, assigned note-taker and specific questions gets a crisp, productive meeting.
- A team that plays back what they heard ("so what we've understood is…") gets a clear yes/no and a correction where they're wrong.
- A team that asks about priority and trade-offs gets a straight, useful answer.
- A team that asks you to design it gets redirected (Section 3.3).

You don't need to tell them which of these they did. The quality of the meeting is the feedback.

4.6 Stay consistent across your sessions

You may run the same conversation with several teams, and other instructors are running it too.

- Keep **light notes** of what you told each team, especially anything from Appendix B and anything you improvised, so team A and team B don't get different stories, and so you can be consistent with yourself in Sprint 10.
- If a team pushes you somewhere Appendix B doesn't cover and it clearly matters, defer it and raise it with the faculty rather than setting a precedent solo.

---

5. Running the first interviews of the day together

For the first meetings on customer day, instructors sit in **as a pair or a small group** for the same team, rather than one-on-one. Two reasons:

1. **Calibration.** You see how a colleague pitches the vagueness, handles "should we use X", and responds to a feature offer, before you're doing it solo.
2. **Consistency.** The first teams through the door get broadly the same customer as the last.

How to run it:

- Agree beforehand who leads. One instructor is the primary voice; the other(s) come in to add a persona's view ("speaking as David for a second…") or to catch a question that went unanswered.
- Don't debate each other in front of the students. If you disagree on an answer, one of you parks it ("let us confirm that and come back to you") and you settle it afterwards, then feed the resolution into Appendix B.
- After the first one or two, debrief for five minutes: what did we say about retention, about segments, about the BR-18 appetite? Lock those in. Then instructors can split off and run the rest solo against the same answers.

---

6. Sprint-by-sprint: what they'll ask, and how to play it

Sprint 2, first meeting (full session). They should drive. Expect wide-ranging, sometimes unfocused questions about the business, the personas and the requirements. Give them the business context generously; stay off solutions. Be a little vague where Section 4.3 invites it. Take at least one feature they offer. If the meeting is disorganised, let it be. The reflection they present to their class afterwards is where that lands.

Sprint 3, data check (10-15 min). Likely topics: how long trade records must be kept (Appendix B: retention), whether "the current position" and "the permanent record" are the same thing to the business (they are not: the audit trail is the firm's legal position), what "reporting" needs (keep it to business terms: cuts, freshness, history). Don't get pulled into relational-vs-NoSQL; that's their decision.

Sprint 4, insight check (10 min). They'll ask which three business insights matter. Answer as Priya / the Sponsor: what you'd actually want to see about the book each month. Appendix B has a steer. Leave the final choice to them; you're confirming direction, not dictating the three.

Sprint 9, UI review (15-20 min). They demo the working interface. Be Joanna: try to place an order, say out loud where you hesitate, where a label confuses you, where you're not sure it worked. Praise what's genuinely clear. An order that's accepted-but-not-yet-filled is the interesting case: react as a client would if it's shown as if something broke. Your reaction is the deliverable; they present it back to their class.

Sprint 10, prioritisation call (10-15 min). If they can't close every gap, they ask what to protect. You've seen their demos, so answer plainly: Musts before Shoulds, a correct balance and an intact audit trail before anything cosmetic. It's legitimate for you to say a particular Should can slip.

Sprint 11, showcase (panel). Different mode. You're shown the finished platform and the reasoning behind the architecture. You can ask whether it meets Sections 8 and 9 of the spec in business terms. Not an interview; don't spring new requirements here.

7. Fielding the questions you'll actually get

They ask…	Play it as…
“Should we use Postgres / Kafka / microservices / AWS?”	“I don’t know what that is. That’s your call. What I need is…”, then restate the outcome.
“How long must we keep records?”	Vague first (“forever, as far as Compliance is concerned”); the Appendix B number if they push.
“What does ‘reporting’ mean?”	Priya’s monthly report to the exec: what’s trading, who’s active. Let them ask for the specifics.
“Which three insights do you want?”	A steer from Appendix B; final choice is theirs to justify.
“Can we add [nice-to-have feature]?”	“Yes please, that sounds great.” Don’t manage the scope for them.
“What client segments?”	Appendix B (by portfolio size and by activity level).
“Market orders only, or limit orders too?”	Appendix B: market at launch is fine; limit orders are a “could”.
“Is [X] a Must or a Should?”	Straight answer, consistent with the spec’s priorities and your earlier sessions.
“Can you look at our schema / API design and tell us if it’s right?”	React as the business (“can it answer David’s dispute question? can Priya run her report without touching live trading?”), but don’t correct the design.
“We didn’t get to prepare, can you just tell us what to build?”	No. Answer what they ask. A thin meeting is a fair outcome.

They ask…	Play it as…
Something genuinely uncovered and material	"Good question, let me confirm and come back to you." Then raise with faculty.

8. Quick reference

Do

- Speak as the business: Joanna, David, Priya, the Sponsor.
- Stay in business language: clients, trades, disputes, the exec committee, Compliance.
- Be vague when a question invites it; make them ask again to pin it down.
- Accept features they offer you, warmly.
- Make them lead the meeting.
- Vary your availability for ad-hoc questions.
- Keep light notes so you're consistent across teams and with yourself later.
- Use Appendix B for anything the spec leaves open.

Don't

- Name a technology, framework, database, pattern or cloud service.
- Design their schema, API or UI, or tell them their approach is right or wrong.
- Mark, coach or rescue; that's their instructor's job, not the customer's.
- Invent hard facts on the spot; defer and check.
- Contradict a requirement that's already written in the spec.
- Fill their silences or upgrade their weak questions before answering.

Appendix A: Pre-interview checklist (for you)

Before the Sprint 2 meeting:

- ☐ Read the Business Requirements Specification once through, at least Sections 2, 3, 6, 8, 9.
- ☐ Read Appendix B of this guide and make sure you're comfortable delivering those answers.
- ☐ Know which team you're the customer for, and which room.
- ☐ Agree with your co-instructor(s) who leads the first meetings.
- ☐ Have a way to take light notes on what you tell each team.

Going in:

- ☐ Let them open. Ask them to walk you through their agenda.
- ☐ Answer in business terms. Redirect every "how should we build it" question.
- ☐ Be a little vague where Section 4.3 applies; accept at least one feature offer.
- ☐ If they don't play back what they heard, don't do it for them.
- ☐ Close on time, even if they haven't covered everything.

Appendix B: Shared answers (faculty to agree before Day 1)

These fill the gaps the spec leaves open on purpose. They are **invented for the exercise**. The point is that every classroom hears the same story, so agree or adjust them as a faculty before customer day, and stick to them. If a team asks something not covered here and it matters, defer and bring it back to the group.

Role	Ask	Numbers / Details
SLO	How long must trade records be retained?	Minimum 7 years after the trade. Build on the assumption records are never deleted or altered. The permanent audit record is separate from the client's current position and remains the firm's legal position even if the live position store is rebuilt.

Role	**Ask**	**Numbers / Details**
Reporting	What does reporting need to show?	Monthly reporting to the executive committee: trading volume by instrument class (equities / FX / crypto) over time; most active instruments in a period; client activity trends (active vs dormant, new vs returning, and activity by segment).
Reporting	How fresh must reporting data be, and how much history is needed?	Updated at least daily; real-time is not required. History goes back to launch. Reporting queries must never compete with live client trading for capacity. Teams choose and justify their final three insights.
Reporting	What client segments should reporting use?	Two cuts are enough: portfolio size (under £10k, £10k–£100k, over £100k) and activity level (active—traded this month, occasional, dormant).
SLO	When must the platform be available?	During the market hours of the instruments covered: UK, US and Indian equity hours; FX nearly 24/5; crypto 24/7. Effectively always on during the trading week. Bank-grade high availability is not expected, but teams should consider and explain the availability challenge.
Client/Trader	Which order types are required at launch?	Market orders are acceptable and sufficient for the Musts. Limit orders are a “could” and may be delivered as a BR-18-style extension; they are not required.
Wildcard	What kind of BR-18 differentiator does the business want?	Open choice. Examples that help Joanna: price alerts, watchlists, a simple performance view, recurring investments. Business-insight options include richer client analytics. The important point is that the team chooses deliberately and explains why it beat the alternatives.

Role	Ask	Numbers / Details
Reporting	What data volumes should be assumed for Sprint 4 synthetic data?	Client base in the low thousands; tens of thousands of orders across the period to date, spread across instruments and client segments, with enough variety to show a trend. Exact numbers are the team's call.
Onboarding/Clients	What identity/onboarding capability is in scope?	Use a fixed set of representative test clients provided by the sign-in fixture. Full onboarding and identity verification are out of scope for this phase.

Appendix C: The personas at a glance

Joanna, self-directed investor (the user). Full-time job, trades her own portfolio around it. Evenings and news events. Wants a trade placed in under a minute and total trust that the screen shows her real position. No jargon, no long confirmation flows, no user guide.

David, head of trading operations (accountable, not a user). Never touches the client UI. Accountable for every trade it produces. For any disputed trade, needs to state exactly what happened and when, from the record, without engineering digging through logs.

Priya, commercial analyst (the reporting consumer). Monthly report to the executive committee on volumes and client activity. Needs reliable history back to launch. Her queries must not compete with live trading.

Programme Sponsor, Head of Digital Distribution (owns the outcome). Owns the business case and the final sign-off. The default voice you speak in when a question isn't specifically Joanna's, David's or Priya's.